

Data Structures

Lecture 4: Asymptotic Analysis

Soobin Um

Mar. 16, 2026

Recap

Algorithm

- **알고리즘**

- 어떤 문제를 해결하기 위한 절차나 방법을 공식화한 형태로 표현한 것

- **좋은 알고리즘의 조건**

- 입력: 외부로부터 하나 이상의 입력값을 받아들일 수 있어야 함
- 출력: 최소 하나 이상의 결과를 산출해야 함
- 명확성: 각 단계는 애매하지 않고 명확하게 기술되어야 함
- 유한성: 유한한 단계 안에 반드시 종료되어야 함
- 효율성: 각 단계는 실행 가능한 연산이어야 함

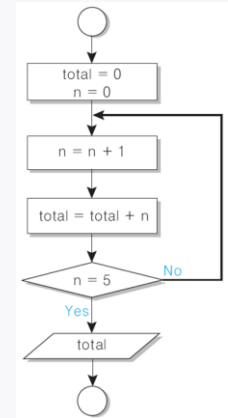
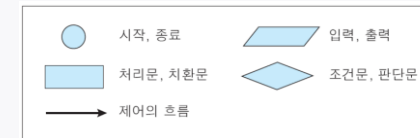
Ways to Represent Algorithms

- **자연어 (natural language)**

- 일상적인 언어로 단계들을 서술하는 방식
- 장점: 직관적이고 이해하기 쉬움
- 단점: 모호성이 생길 수 있음

- **순서도 (flow chart)**

- 기호를 이용해 알고리즘의 흐름을 시각적으로 표현
- 장점: 전체 구조와 흐름을 한눈에 파악 가능
- 단점: 복잡한 알고리즘의 경우, 표현이 어려울 수 있음



- **의사코드 (pseudo-code)**

- 실제 프로그래밍 언어와 유사한 형식으로, 언어에 구애받지 않고 작성
- 장점: 실제 구현에 가깝고 명확성 확보 가능
- 단점: 완전히 표준화된 문법이 없음

```
fibonacci(n)
  if (n < 0) then
    stop
  if (n ≤ 1) then
    return n
  f1 ← 0
  f2 ← 1
  for (i in {2, 3, ..., n}) do
    fn ← f1 + f2
    f1 ← f2
    f2 ← fn
  return fn
```

- **프로그래밍 언어 (programming language)**

- 실제 프로그래밍 언어로 구현
- 장점: 실행 가능하고 테스트 가능
- 단점: 언어 문법에 따라 가독성이 떨어질 수 있음

```
def factorial(n):
  if n==0:
    return 1
  else:
    return n*factorial(n-1)
```

Goals of Designing Algorithms

- 이해하기 쉽고, 구현하기 쉽고, 디버깅하기 쉬운 알고리즘 설계
 - 프로그램을 혼자서만 사용하는 경우는 드뭄. 협업/유지보수 고려가 필수적
 - 읽기 어렵고 구조가 나쁜 코드는 유지보수 비용을 크게 증가시킴
- 컴퓨터 자원을 효율적으로 사용하는 (효율성 높은) 알고리즘 설계
 - CPU, 메모리, 저장장치 등의 자원은 한정적임
 - 같은 문제라도 자료구조 선택에 따라 실행 시간이 수십 배까지 차이가 날 수 있음



Asymptotic Analysis

Theoretical Analysis

- **수행 시간 함수 $T(n)$**

- 알고리즘의 수행 시간은 일반적으로 입력 크기 n 의 함수로 표현 가능

$$T(n) = \text{기본 연산의 실행 횟수}$$

- **기본 연산 (basic operation)**

- 알고리즘의 수행 시간을 결정하는 핵심 연산
- 수행 시간이 고정된 연산을 가정:
 - 단순 할당 (assignment)
 - 산술 연산 (+, -, $\times$, $\div$)
 - 비교 연산 (<, >, =)
 - 배열 원소 접근
- 반복문: (내부 기본 연산 횟수 $\times$ 반복 횟수)로 계산

Example 1: 피보나치 수열

- 피보나치 수열의 n 번째 항을 구하는 의사코드

fibonacci(n)	$n < 0$	$n = 0$	$n = 1$	$n > 1$
01 if (n<0) then	1	1	1	1
02 stop	1	0	0	0
03 if (n≤1) then	0	1	1	1
04 return n	0	1	1	0
05 f1 ← 0	0	0	0	1
06 f2 ← 1	0	0	0	1
07 for (i in {2, 3, ..., n}) do	0	0	0	n-1
08 fn ← f1 + f2	0	0	0	n-1
09 f1 ← f2	0	0	0	n-1
10 f2 ← fn	0	0	0	n-1
11 return fn	0	0	0	1
T(n) =	2	3	3	4n+1

Example 2: summation

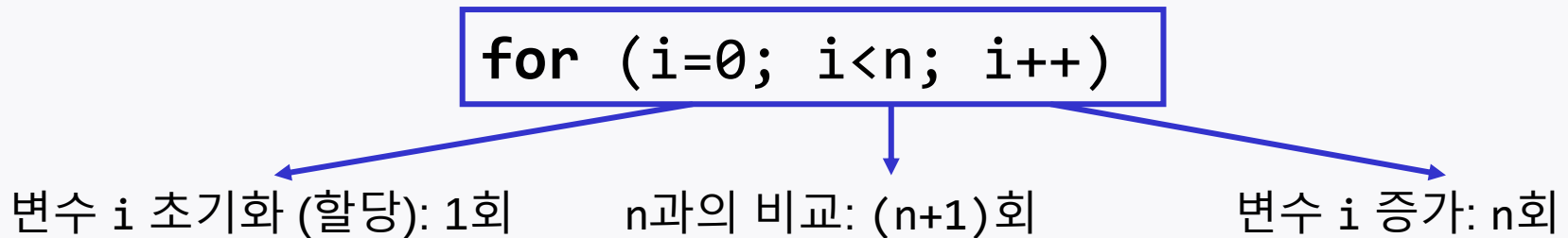
- n 개 숫자들의 합을 구하는 프로그래밍 언어

<code>int sum(int list[], int n)</code>	# of steps
01 <code>int i;</code>	0
02 <code>int temp_sum = 0;</code>	1
03 <code>for (i=0; i<n; i++)</code>	$1+(n+1)+n$
04 <code>temp_sum += list[i];</code>	$2n$
05 <code>return temp_sum;</code>	1
$T(n) =$	$4n+4$

Example 2: summation

- n 개 숫자들의 합을 구하는 프로그래밍 언어

<code>int sum(int list[], int n)</code>	# of steps
01 <code>int i;</code>	0
02 <code>int temp_sum = 0;</code>	1
03 <code>for (i=0; i<n; i++)</code>	$1+(n+1)+n$
04 <code>temp_sum += list[i];</code>	$2n$
05 <code>return temp_sum;</code>	1
$T(n) =$	$4n+4$



Asymptotic Growth Function

- 점근적 성장 함수 $f(n)$
 - $T(n)$ 을 단순화한 결과로, 입력 크기 n 에 따라 실행 시간이 어떤 차수로 성장하는지를 나타냄

Asymptotic Growth Function

- 점근적 성장 함수 $f(n)$
 - $T(n)$ 을 단순화한 결과로, 입력 크기 n 에 따라 실행 시간이 어떤 차수로 성장하는지를 나타냄

$$\text{e.g., } T(n) = 3n^2 + 2n + 5 \rightarrow f(n) = n^2$$

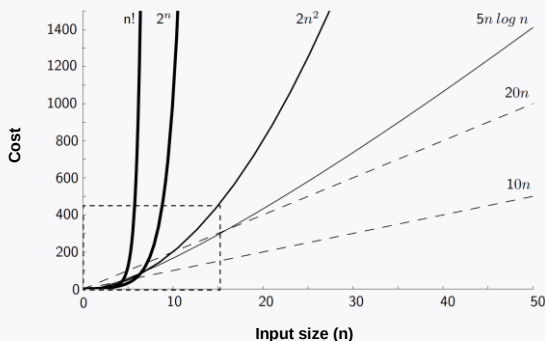
Asymptotic Growth Function

- 점근적 성장 함수 $f(n)$

- $T(n)$ 을 단순화한 결과로, 입력 크기 n 에 따라 실행 시간이 어떤 차수로 성장하는지를 나타냄

e.g., $T(n) = 3n^2 + 2n + 5 \rightarrow f(n) = n^2$

$\log n$	<	n	<	$n \log n$	<	n^2	<	n^3	<	2^n
0		1		0		1		1		2
1		2		2		4		8		4
2		4		8		16		64		16
3		8		24		64		512		256
4		16		64		256		4096		65536
5		32		160		1024		32768		4294967296



Asymptotic Analysis

- 점근적 시간 복잡도 표기법

- 수행 시간 함수 $T(n)$ 과 단순화된 성장 함수 $f(n)$ 사이의 관계를 상한, 하한, 정확한 차수라는 서로 다른 관점에서 표현하는 방법

Big-O $O(f(n))$

실행 시간이 최대 어느 정도까지 걸리는지 (상한)

" $f(n)$ 보다 더 느리게 동작하지는 않는다"

Big-Ω $\Omega(f(n))$

실행 시간이 최소 어느 정도는 걸린다는 것 (하한)

"아무리 잘 되어도 $f(n)$ 보다 빠르지는 않다"

Big-Θ $\Theta(f(n))$

실행 시간이 $f(n)$ 과 같은 차수로 증가 (상/하한 동시)

"상/하한이 같은 차수: 실행 시간의 정확한 차수를 전달"

Big-O

- 빅-오 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배 즉, $c \cdot f(n)$ 을 초과하지 않음
- $T(n)$ 은 $f(n)$ 보다 빠르게 증가하지 않음 $\rightarrow$ 점근적 상한 (upper bound)

Big-O

- 빅-O 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배 즉, $c \cdot f(n)$ 을 초과하지 않음
- $T(n)$ 은 $f(n)$ 보다 빠르게 증가하지 않음 $\rightarrow$ 점근적 상한 (upper bound)

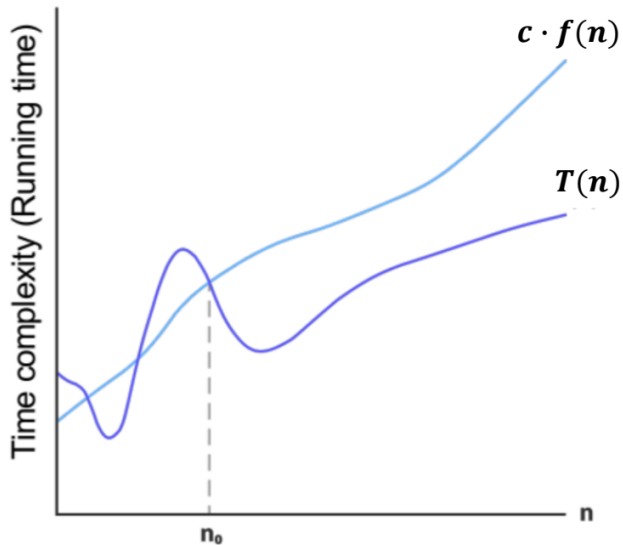
$T(n) \in O(f(n))$ iff there exist positive constants c and n_0
such that $T(n) \leq c \cdot f(n)$ for all $n > n_0$

Big-O

• 빅-O 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배 즉, $c \cdot f(n)$ 을 초과하지 않음
- $T(n)$ 은 $f(n)$ 보다 빠르게 증가하지 않음 $\rightarrow$ 점근적 상한 (upper bound)

$T(n) \in O(f(n))$ iff there exist positive constants c and n_0 such that $T(n) \leq c \cdot f(n)$ for all $n > n_0$



$T(n)$ 은 $f(n)$ 보다 더 빠르게 증가하지 않음
 $\rightarrow T(n)$ 은 $O(f(n))$ 에 속함

Big-O 예제

- **$T(n) = 7n + 3 \in O(n)$**
 - $n > n_0$ 에 대해, $7n + 3 \leq cn$ 을 만족하는 c 와 n_0 를 찾아야 함

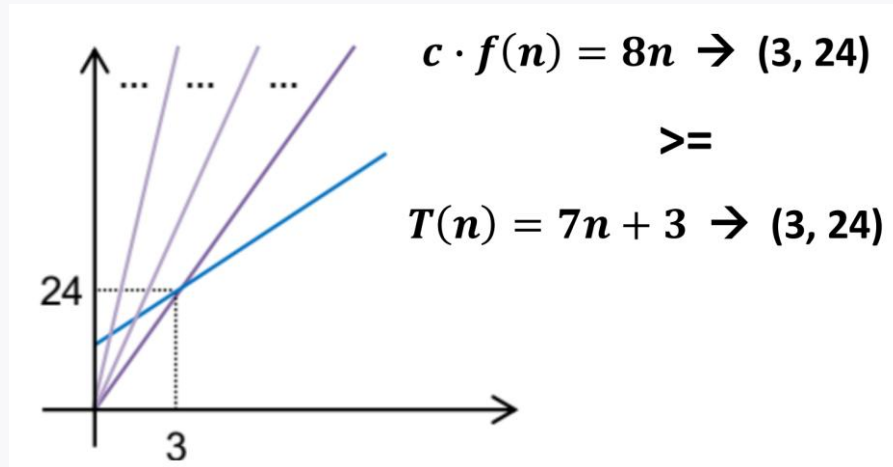
Big-O 예제

- **$T(n) = 7n + 3 \in O(n)$**
 - $n > n_0$ 에 대해, $7n + 3 \leq cn$ 을 만족하는 c 와 n_0 를 찾아야 함
e.g., $c=8, n_0=3$ / $c=10, n_0=1$ / $c=20, n_0=1$...

Big-O 예제

- $T(n) = 7n + 3 \in O(n)$

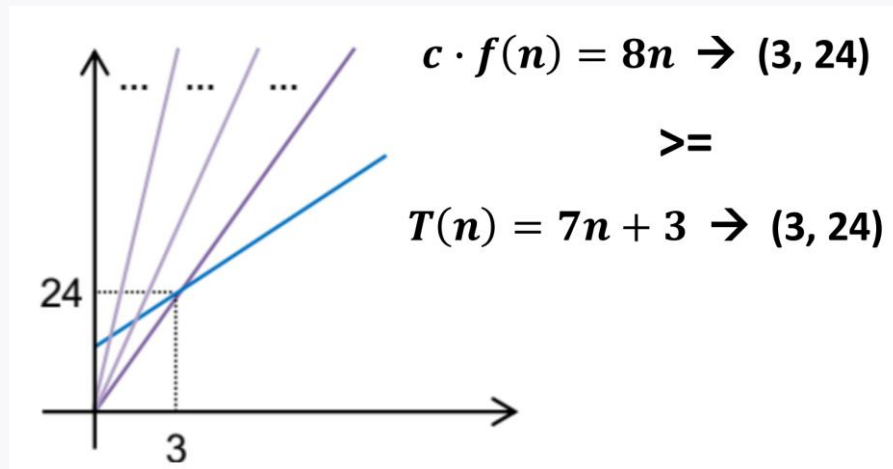
- $n > n_0$ 에 대해, $7n + 3 \leq cn$ 을 만족하는 c 와 n_0 를 찾아야 함
e.g., $c=8, n_0=3$ / $c=10, n_0=1$ / $c=20, n_0=1$...



Big-O 예제

- **$T(n) = 7n + 3 \in O(n)$**

- $n > n_0$ 에 대해, $7n + 3 \leq cn$ 을 만족하는 c 와 n_0 를 찾아야 함
e.g., $c=8, n_0=3$ / $c=10, n_0=1$ / $c=20, n_0=1$...



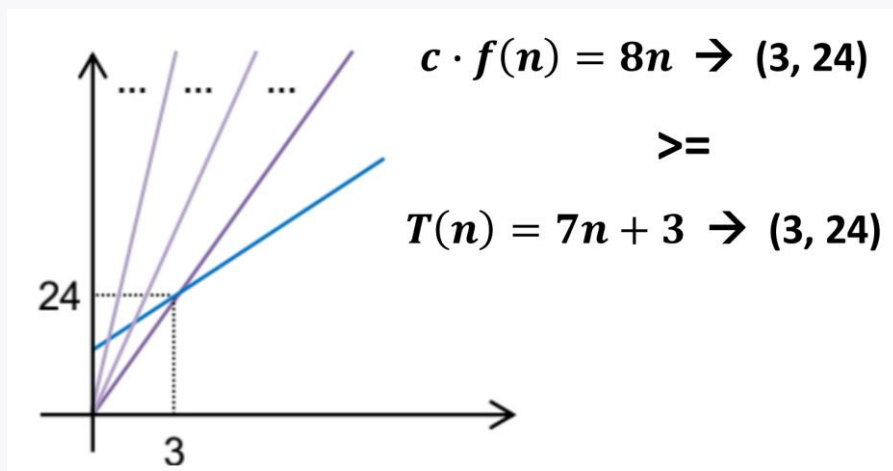
- **$T(n) = 3n^3 + 7n + 5 \in O(n^3)$**

- $n > n_0$ 에 대해, $3n^3 + 7n + 5 \leq cn^3$ 을 만족하는 c 와 n_0 를 찾아야 함

Big-O 예제

- **$T(n) = 7n + 3 \in O(n)$**

- $n > n_0$ 에 대해, $7n + 3 \leq cn$ 을 만족하는 c 와 n_0 를 찾아야 함
e.g., $c=8, n_0=3$ / $c=10, n_0=1$ / $c=20, n_0=1$...



- **$T(n) = 3n^3 + 7n + 5 \in O(n^3)$**

- $n > n_0$ 에 대해, $3n^3 + 7n + 5 \leq cn^3$ 을 만족하는 c 와 n_0 를 찾아야 함
e.g., 모든 $n > n_0 = 1$ 에 대해, $T(n)$ 은 $(3 + 7 + 5)n^3$ 보다 크지 않음
(즉, $c=15$ and $n_0=1$)

다양한 형태의 연산들에 대한 Big-O 분석

Loop	# of steps
01 <code>int m = 0;</code>	1
02 <code>for (int i=0; i<n; i++)</code>	$1+(n+1)+n$
03 <code>m = m + 1;</code>	$2n$
$T(n) =$	$4n+3$
$T(n) \in O(n)$	

다양한 형태의 연산들에 대한 Big-O 분석

Loop	# of steps
01 <code>int m = 0;</code>	1
02 <code>for (int i=0; i<n; i++)</code>	$1+(n+1)+n$
03 <code>m = m + 1;</code>	$2n$
$T(n) =$	$4n+3$
$T(n) \in O(n)$	

Nested loop	# of steps
01 <code>int m = 0;</code>	1
02 <code>for (int i=0; i<n; i++)</code>	$1+(n+1)+n$
03 <code>for (int j=0; j<n; j++)</code>	$n(2n+2)$
04 <code>m = m + 1;</code>	$2n \cdot n$
$T(n) =$	$4n^2+4n+3$
$T(n) \in O(n^2)$	

다양한 형태의 연산들에 대한 Big-O 분석

Consecutive statements	# of steps
01 <code>int a = 0, b = 0;</code>	2
02 <code>a = a + 3;</code>	2
03 <code>for (int i=0; i<n; i++)</code>	$1+(n+1)+n$
04 <code> a = a + 2;</code>	$2n$
05 <code>for (int i=0; i<n; i++)</code>	$1+(n+1)+n$
06 <code> for (int j=0; j<n; j++)</code>	$n(2n+2)$
07 <code> b = b + 2;</code>	$2n \cdot n$
$T(n) =$	$4n^2+7n+8$
$T(n) \in O(n^2)$	

다양한 형태의 연산들에 대한 Big-O 분석

If-then-else statements	# of steps
01 <code>int m = 0;</code>	1
02 <code>if (n==0)</code>	1
03 <code>return false;</code>	1
04 <code>else {</code>	0
05 <code>for (int i=0; i<n; i++)</code>	$1+(n+1)+n$
06 <code>m = m + i;</code>	$2n$
07 <code>}</code>	0
$T(n) =$	3 or $4n+4$
$T(n) \in O(n)$	

Prefix Averages

- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해
앞에서부터의 평균을 계산

$$A = [5, 3, 1, 7, 9]$$

Prefix Averages

- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해 앞에서부터의 평균을 계산

$$A = [5, 3, 1, 7, 9]$$



$$\text{PreAve} = [5$$

Prefix Averages

- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해 앞에서부터의 평균을 계산

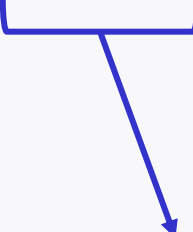
$$A = [5, 3, 1, 7, 9]$$


$$\text{PreAve} = [5, 4]$$

Prefix Averages

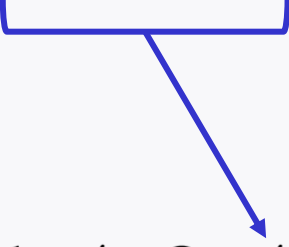
- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해 앞에서부터의 평균을 계산

$$A = [5, 3, 1, 7, 9]$$


$$\text{PreAve} = [5, 4, 3]$$

Prefix Averages

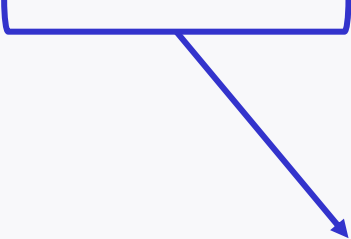
- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해 앞에서부터의 평균을 계산

$$A = [5, 3, 1, 7, 9]$$


$$\text{PreAve} = [5, 4, 3, 4]$$

Prefix Averages

- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해 앞에서부터의 평균을 계산

$$A = [5, 3, 1, 7, 9]$$


$$\text{PreAve} = [5, 4, 3, 4, 5]$$

Prefix Averages

- 길이 n 의 배열이 주어졌을 때, 각 위치 i 에 대해 앞에서부터의 평균을 계산

$$A = [5, 3, 1, 7, 9]$$

$$\text{PreAve} = [5, 4, 3, 4, 5]$$

$$\text{PreAve}[j] := \frac{\sum_{i=0}^j A[i]}{j+1}$$

Prefix Averages

- 구현 1

- 배열 A 상에서의 각각의 i 마다, $0 \sim i$ 까지의 합산을 구한 뒤 평균

Prefix Averages (Naïve)	# of steps
01 for (int i=0; i<n; i++) { 02 int sum = 0; 03 for (int j=0; j<i+1; j++) 04 sum += A[j]; 05 PreAve[i] = sum / (i+1); 06 }	
$T(n) =$	
$T(n) \in O(n^2)$	

Prefix Averages

- 구현 2

- 누적합을 이용해 한번의 덧셈 연산만으로 평균 계산 수행

Prefix Averages (Efficient)	# of steps
01 <code>int sum = 0;</code> 02 <code>for (int i=0; i<n; i++) {</code> 03 <code> sum += A[i];</code> 04 <code> PreAve[i] = sum / (i+1);</code> 05 <code>}</code>	
$T(n) =$	
$T(n) \in O(n)$	

Big-Ω

- 빅-오메가 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배, 즉, $c \cdot f(n)$ 이상임
- $T(n)$ 은 $f(n)$ 보다는 빠르게 증가함을 보여주는 점근적 하한 (lower bound) 표현

Big-Ω

- 빅-오메가 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배, 즉, $c \cdot f(n)$ 이상임
- $T(n)$ 은 $f(n)$ 보다는 빠르게 증가함을 보여주는 점근적 하한 (lower bound) 표현

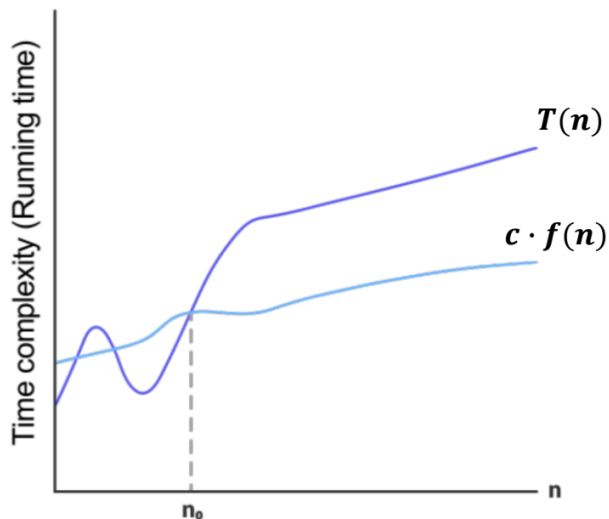
$T(n) \in \Omega(f(n))$ iff there exist positive constants c and n_0 such that $T(n) \geq c \cdot f(n)$ for all $n > n_0$

Big-Ω

• 빅-오메가 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 일정한 상수배, 즉, $c \cdot f(n)$ 이상임
- $T(n)$ 은 $f(n)$ 보다는 빠르게 증가함을 보여주는 점근적 하한 (lower bound) 표현

$T(n) \in \Omega(f(n))$ iff there exist positive constants c and n_0 such that $T(n) \geq c \cdot f(n)$ for all $n > n_0$



$T(n)$ 은 $f(n)$ 보다 더 빠르게 증가함
→ $T(n)$ 은 $\Omega(f(n))$ 에 속함

Big- Θ

- 빅-세타 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 두 상수배 사이에 위치: $c_1 f(n) \leq T(n) \leq c_2 f(n)$
- $T(n)$ 은 $f(n)$ 과 같은 차수로 증가함을 보여주는 점근적 tight bound 표현

Big- Θ

- 빅-세타 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 두 상수배 사이에 위치: $c_1 f(n) \leq T(n) \leq c_2 f(n)$
- $T(n)$ 은 $f(n)$ 과 같은 차수로 증가함을 보여주는 점근적 tight bound 표현

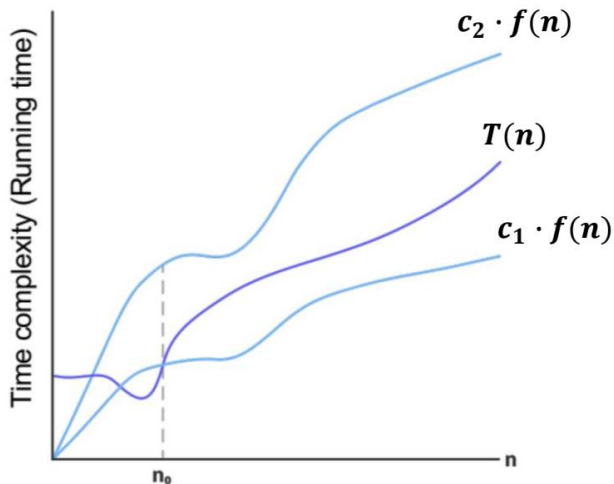
$T(n) \in \Theta(f(n))$ iff there exist positive constants c_1 , c_2 and n_0 such that $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n > n_0$

Big-Θ

• 빅-세타 표기법

- 충분히 큰 입력 크기 $n > n_0$ 에 대해, 알고리즘의 수행 시간 $T(n)$ 은 $f(n)$ 의 두 상수배 사이에 위치: $c_1 f(n) \leq T(n) \leq c_2 f(n)$
- $T(n)$ 은 $f(n)$ 과 같은 차수로 증가함을 보여주는 점근적 tight bound 표현

$T(n) \in \Theta(f(n))$ iff there exist positive constants c_1 , c_2 and n_0 such that $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n > n_0$



$T(n)$ 은 $f(n)$ 과 같은 속도로 증가함
→ $T(n)$ 은 $\Theta(f(n))$ 에 속함

Look ahead

리스트(List)